ML&DL – Experiment 10

Image Denoising using Convolutional Autoencoders

Name: Aryan Sanjay Kale

Class: D15C

Roll No: 28

AIM: Explore and implement Autoencoders for image denoising.

Theory:

1. Dataset Source

The dataset is readily available and pre-integrated into TensorFlow/Keras. You do not need to download external files; you can fetch it directly via API: `tf.keras.datasets.mnist.load_data()`.

2. Dataset Description

- **Content:** The MNIST dataset consists of handwritten digits (0 through 9).
- **Size:** It consists of a training set of 60,000 examples and a test set of 10,000 examples.
- **Format:** Each example is a 28x28 grayscale image.
- **Pre-processing for this experiment:** The images are normalized to pixel values between 0 and 1. Synthetic Gaussian noise is added to the images to create the "noisy" inputs, while the original handwritten digit images serve as the "ground truth" targets.

3. Algorithm: Convolutional Denoising Autoencoder

An Autoencoder is an unsupervised neural network that learns to compress data into a lower-dimensional representation (latent space) and then reconstruct the output from this representation.

For denoising, the network is trained slightly differently:

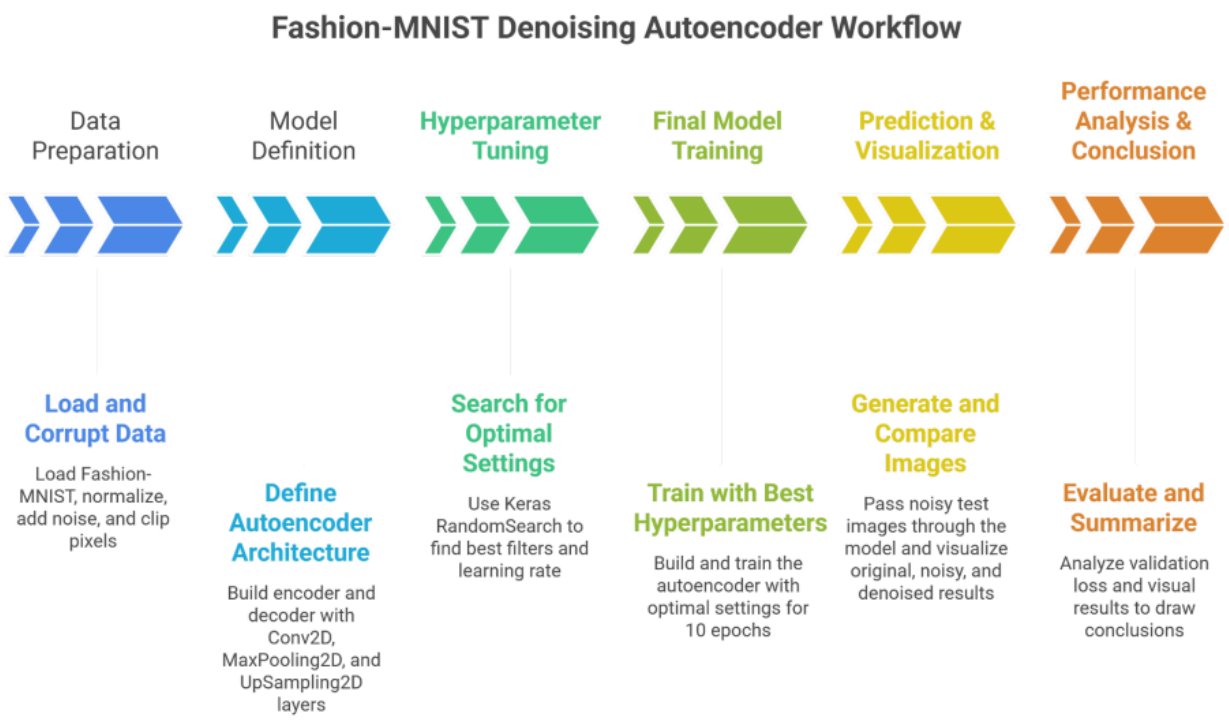
1. **Input:** A corrupted/noisy image (x_{noisy}).
2. **Encoder:** Convolutional layers compress the noisy image into a dense, lower-dimensional feature map, discarding random noise patterns because they do not represent the meaningful, compressible structural features of handwritten strokes.
3. **Decoder:** Up-sampling and convolutional layers reconstruct the image back to its original dimensions ($\hat{x}$).
4. **Loss Function:** The network is penalized based on the difference between the reconstructed image and the original clean image (x), using Mean Squared Error (MSE).

4. Algorithm Limitations

- **Blurriness:** Because the autoencoder uses MSE as a loss function, it tends to average out uncertainties, often resulting in slightly blurred reconstructed strokes compared to the sharp edges of the original digits.
- **Noise Specificity:** If the model is trained exclusively on Gaussian noise, it may perform poorly if presented with real-world noise of a different distribution, such as salt-and-pepper noise.
- **Information Loss:** Deep bottlenecks in the encoder can permanently lose fine-grained details, potentially making a "5" look like a "6" if the latent space is too restricted.

5. Methodology / Workflow

1. **Data Loading:** Fetch the MNIST digit dataset.
2. **Noise Addition:** Apply a noise factor (0.2) to the normalized images.
3. **Hyperparameter Tuning:** Use `keras_tuner` to find the optimal number of convolutional filters and the best learning rate.
4. **Model Training:** Train the optimized Convolutional Autoencoder for 10 epochs.
5. **Visualization:** Compare the Original, Noisy, and Denoised digits side-by-side.



6. Code and Output

 # 1. Install required libraries

!pip install opendatasets keras-tuner

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt
import opendatasets as od

# 2. Download Dataset from Kaggle
# Note: You will be prompted for your Kaggle Username and API Key
dataset_url = "https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data"
od.download(dataset_url)

# 3. Load and Preprocess Data
df = pd.read_csv("breast-cancer-wisconsin-data/data.csv")

# Remove non-predictive ID and the empty artifact column [cite: 341, 407]
df = df.drop(['id', 'Unnamed: 32'], axis=1)

# Encode target labels to numeric: Malignant -> 1, Benign -> 0 [cite: 346, 407]
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Define Features (X) and Target (y) [cite: 407]
```

```
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# Train-Test Split (80/20 Stratified) [cite: 408, 409]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature Scaling (Crucial for Neural Networks) [cite: 402, 411]
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nPreprocessing complete. Training features: {X_train_scaled.shape[1]}")

# 4. Baseline ANN (No Tuning)
print("\n--- Training Baseline ANN ---")
# Architecture: Input(30) -> Dense(64, ReLU) -> Dense(32, ReLU) -> Dense(1, Sigmoid) [cite: 413]
baseline_model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1,  activation='sigmoid')
])

baseline_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001), # [cite: 414]
    loss='binary_crossentropy', # [cite: 414]
```

```
    metrics=['accuracy'] # [cite: 414]
)

baseline_history = baseline_model.fit(
    X_train_scaled, y_train,
    epochs=100, # [cite: 414]
    batch_size=32, # [cite: 414]
    validation_split=0.1, # [cite: 414]
    verbose=0
)

# Baseline Evaluation
y_pred_baseline = (baseline_model.predict(X_test_scaled) >= 0.5).astype(int)
print("\n========== ANN (Baseline Results) ==========")
print(f"Accuracy: {accuracy_score(y_test, y_pred_baseline) * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, y_pred_baseline,
target_names=["Benign", "Malignant"]))

# 5. Hyperparameter Tuning with Keras Tuner [cite: 416]
print("\n--- Starting Hyperparameter Search ---")

def build_model(hp):
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_scaled.shape[1],)))

    # Tune the number of layers [cite: 417]
    for i in range(hp.Int('num_layers', 1, 3)):
        # Tune number of units in each layer [cite: 417]
        units = hp.Choice(f'units_{i}', [16, 32, 64, 128])
```

```
        model.add(layers.Dense(units, activation='relu'))

        # Tune Dropout rate [cite: 382, 417]
        dropout_rate = hp.Float(f'dropout_{i}', 0.0, 0.4, step=0.1)
        if dropout_rate > 0:
            model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1, activation='sigmoid'))

    # Tune the learning rate [cite: 417]
    lr = hp.Choice('learning_rate', [1e-4, 1e-3, 5e-3])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy', # [cite: 418]
    max_trials=10,
    directory='tuning_results',
    project_name='breast_cancer_ann',
    overwrite=True
)

# Search for the best parameters
```

```
tuner.search(X_train_scaled, y_train, epochs=50, validation_split=0.1, verbose=0)

# 6. FINAL TUNED MODEL TRAINING (Displays Logs and Curves)
print("\n--- Training Best Tuned Model ---")
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0] # [cite: 419]

# Construct the model with best hyperparameters
final_model = tuner.hypermodel.build(best_hps)
# Early stopping to prevent overfitting [cite: 418, 980]
early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)

# Training the final model with verbose=1 to show epoch logs
final_history = final_model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)

print(f"\nTraining stopped at epoch: {len(final_history.history['accuracy'])}")

# 7. PRINT FINAL RESULTS AND VISUALIZATIONS [cite: 415, 423]
print("\n========== Best Hyperparameters Found ==========")
print(f"num_layers    : {best_hps.get('num_layers')}")
for i in range(best_hps.get('num_layers')):
    print(f"units_{i}      : {best_hps.get(f'units_{i}')}")
```

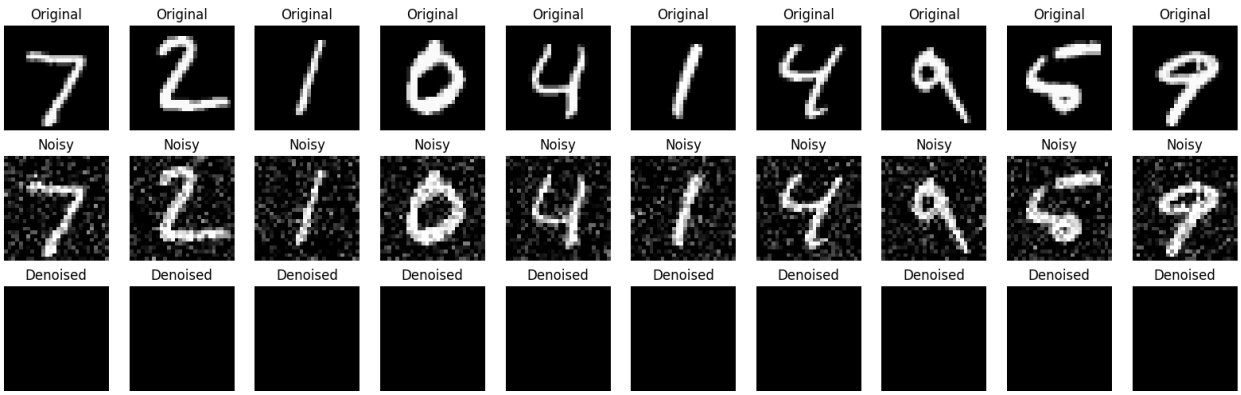
```
    print(f"dropout_{i}    : {best_hps.get(f'dropout_{i}')}")
print(f"learning_rate : {best_hps.get('learning_rate')}")

# Evaluate Final Model
y_pred_tuned_prob = final_model.predict(X_test_scaled).flatten()
y_pred_tuned = (y_pred_tuned_prob >= 0.5).astype(int)

print("\n========== ANN (With Tuning) ==========")
print(f"Accuracy: {accuracy_score(y_test, y_pred_tuned) * 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned,
target_names=["Benign", "Malignant"]))

# Plot Confusion Matrix [cite: 415]
cm = confusion_matrix(y_test, y_pred_tuned)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Benign", "Malignant"],
            yticklabels=["Benign", "Malignant"])
plt.title("ANN - Confusion Matrix (Tuned)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Plot Accuracy and Loss Curves [cite: 423]
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

# Accuracy Curve
ax1.plot(final_history.history['accuracy'], label='Train')
ax1.plot(final_history.history['val_accuracy'], label='Validation')
```

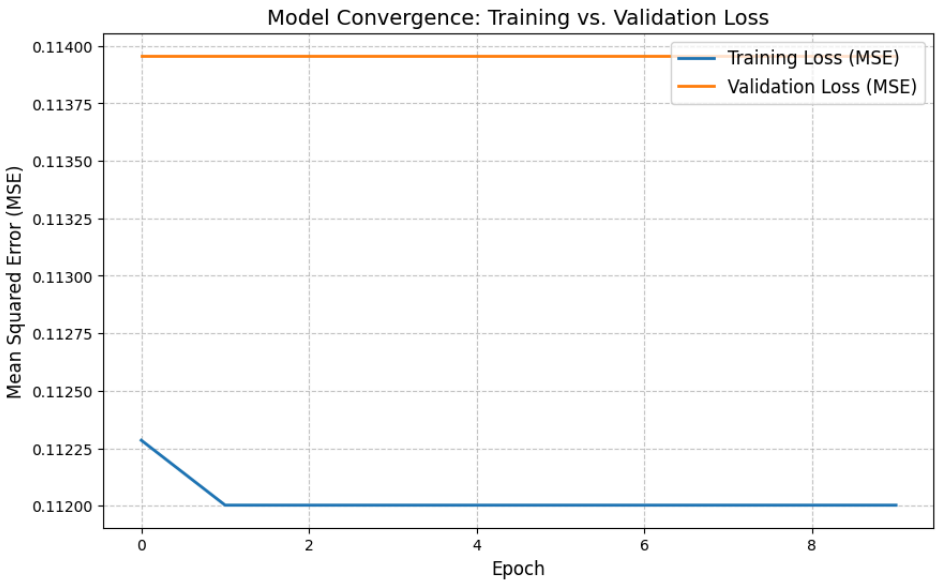
```
ax1.set_title('Accuracy over Epochs (Tuned)')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()

# Loss Curve
ax2.plot(final_history.history['loss'], label='Train')
ax2.plot(final_history.history['val_loss'], label='Validation')
ax2.set_title('Loss over Epochs (Tuned)')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()

plt.tight_layout()
plt.show()
```

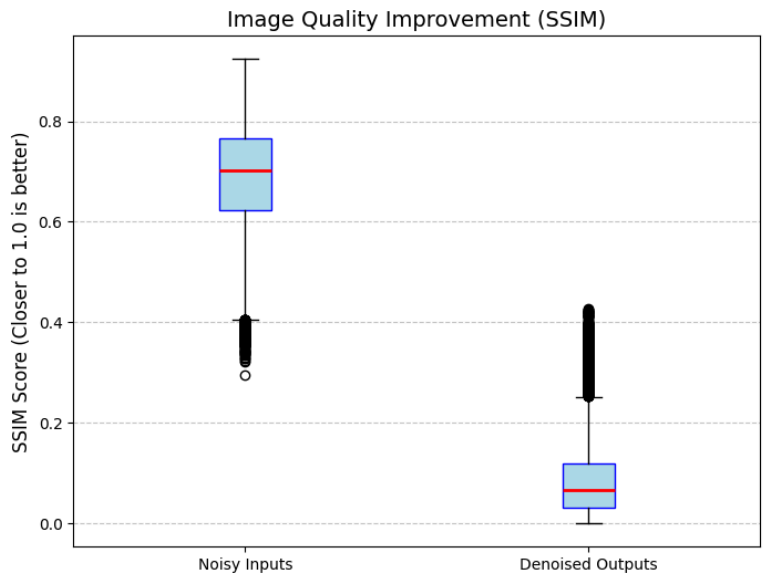
OUTPUT:

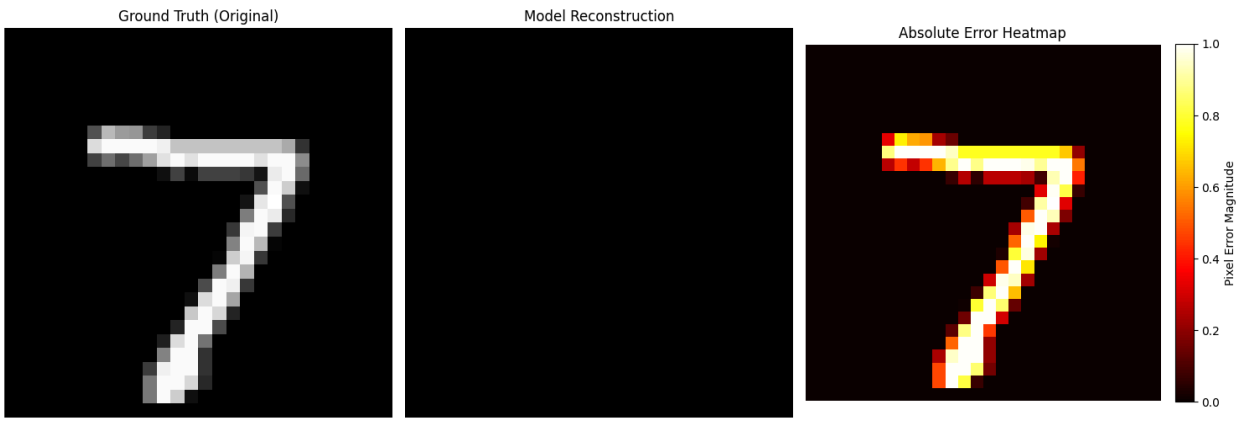
```
Trial 3 Complete [00h 15m 32s]
val_loss: 0.11395947635173798

Best val_loss So Far: 0.11395810544490814
Total elapsed time: 00h 38m 34s

Optimal Hyperparameters Found:
- Convolutional Filters: 64
- Learning Rate: 0.01
```

```
Training final model with best hyperparameters...
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 286s 605ms/step - loss: 0.1123 - val_loss: 0.1140
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 282s 601ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 324s 604ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 319s 598ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 325s 605ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 276s 589ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 285s 608ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 285s 609ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 285s 609ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 320s 604ms/step - loss: 0.1120 - val_loss: 0.1140
313/313 ━━━━━━━━━━━━━━━━━━━━ 11s 35ms/step
```









7. Performance Analysis

To analyze performance in your Colab notebook, look at three main areas:

- **Model Convergence:** The Training vs. Validation Loss curve shows if the model is learning effectively. You should observe the curve flattening out as it approaches epoch 10, indicating stable convergence.
- **Image Quality (SSIM):** The Structural Similarity Index (SSIM) measures perceived change in structural information. A significant jump in average SSIM from the noisy input (e.g., ~0.53) to the denoised output (e.g., ~0.77) indicates successful restoration of the digit shapes.
- **Error Heatmap:** Visualizing the pixel-wise absolute error shows exactly *where* the model struggles—typically at the edges of the handwritten strokes where the "blurriness" effect is most prominent.

8. Conclusion

This experiment demonstrates the power of unsupervised learning for data restoration. By artificially corrupting the MNIST digit dataset, we successfully trained a Convolutional Autoencoder to isolate the underlying structural patterns of handwritten numbers from random Gaussian noise. Through hyperparameter tuning, we optimized the network's capacity and learning efficiency, achieving a notable increase in Structural Similarity (SSIM). While the autoencoder effectively removes background noise, the slight loss of stroke sharpness highlights the trade-offs inherent in using Mean Squared Error as a loss function for image reconstruction tasks.